

DESARROLLO WEB INTEGRADO

Implementación de API REST

Sesión 4

Ciclo VII: Agosto 2026

Mg. Ing. Richard Doann ASENCIOS VILCHEZ



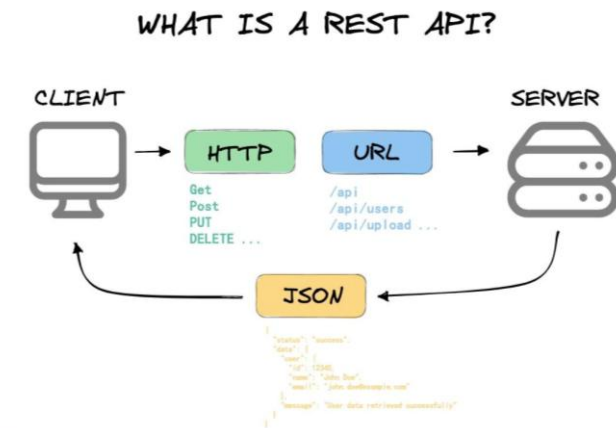
Universidad
Tecnológica
del Perú

Unidad 1 API REST



inicio

**“Una API REST no solo
expone datos: define
cómo diferentes
sistemas pueden
comunicarse de manera
ordenada, segura y
predecible”**



INICIO

Dudas sobre la sesión anterior



INICIO

Dudas sobre la sesión anterior

Propósito de las Pruebas de Software

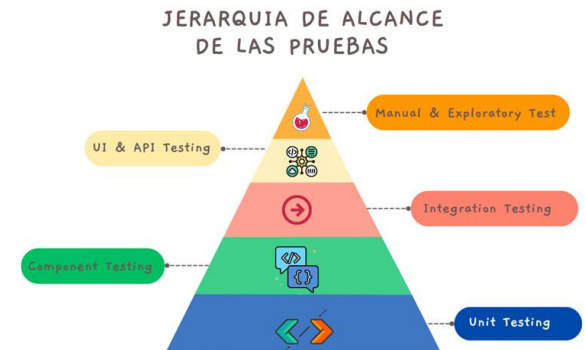


Test-drive development (TDD)

TEST-DRIVEN DEVELOPMENT



Jerarquía de Alcance de las Pruebas



Conocimientos Previos



1.- ¿Cuál es la secuencia correcta del ciclo de TDD?.

- a. Refactor → Test → Code
- b. Code → Test → Refactor
- c. Test → Code → Refactor
- d. Test → Refactor → Code

2.- En el contexto de Spring Boot, ¿qué herramienta se utiliza comúnmente para realizar pruebas unitarias?.

- a. JIRA
- b. Junit
- c. Docker
- d. Maven

3.- ¿Cuál de las siguientes afirmaciones describe mejor el objetivo de TDD?

- a. Escribir pruebas solo después de implementar funcionalidades.
- b. Minimizar el uso de pruebas en desarrollo.
- c. Escribir código guiado por pruebas previamente diseñadas.
- d. Automatizar despliegues continuos.



Logro de la sesión:



Al finalizar la sesión el estudiante conocerá la forma de implementar una **API REST** básica en Spring Boot, uso correcto de los métodos HTTP (GET, POST, PUT, DELETE) de acuerdo con la semántica **REST**; pruebas y validaciones de los endpoints creados mediante herramientas como **Postman**, a través de casos prácticos.



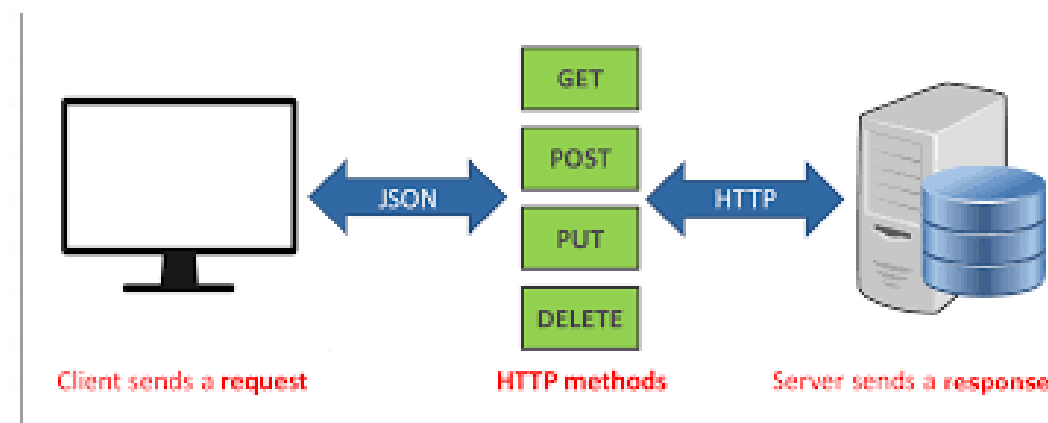
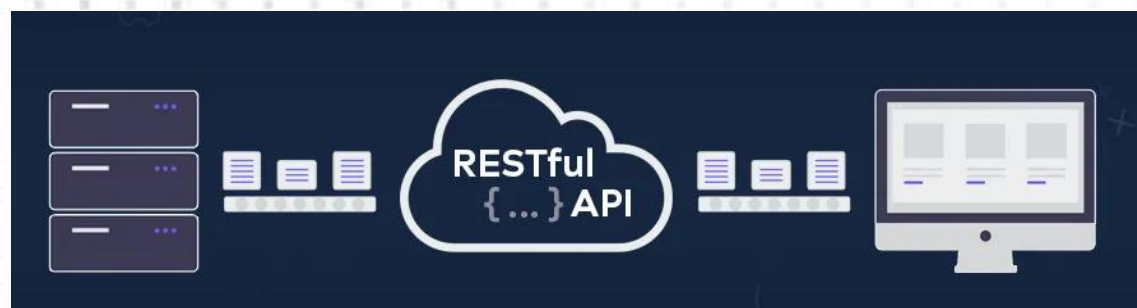


Este tema servirá para entender y definir las API REST, utilizándolas para conectar aplicaciones web con servicios y datos externos, lo que permite crear servicios web y fomentar la interoperabilidad entre plataformas.



¿De que otra forma podemos utilizar lo aprendido?

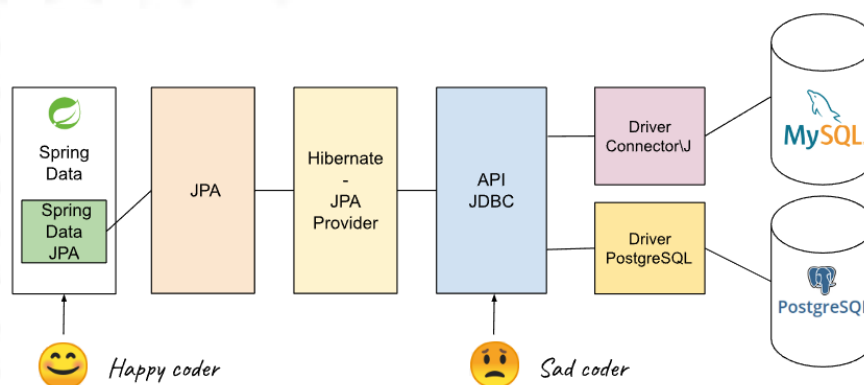
¿Para que será útil lo que estamos aprendiendo en esta sección?



UTILIDAD

TEMA:

Implementación de API REST



Introducción a las API REST



¿Qué es una API?

- API (Application Programming Interface): Es un **conjunto de reglas y definiciones** que permite que dos aplicaciones distintas puedan **comunicarse entre sí**.
- Ejemplo cotidiano: cuando una aplicación móvil de delivery muestra restaurantes y pedidos, realmente está consultando una API que envía datos desde un servidor hacia el cliente.

¿Qué es REST?

- REST (Representational State Transfer): Es un **estilo de arquitectura** para diseñar servicios web.
- Propone un conjunto de principios que hacen que la comunicación sea más ligera, escalable y fácil de usar.
- Fue definido por Roy Fielding en el año 2000..
- Las API REST utilizan **HTTP** como protocolo base para comunicar cliente ↔ servidor.



TRANSFORMACIÓN

Introducción a las API REST

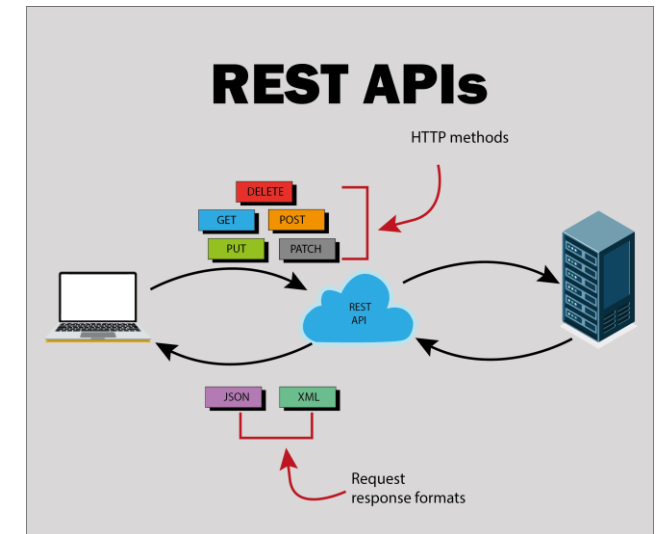
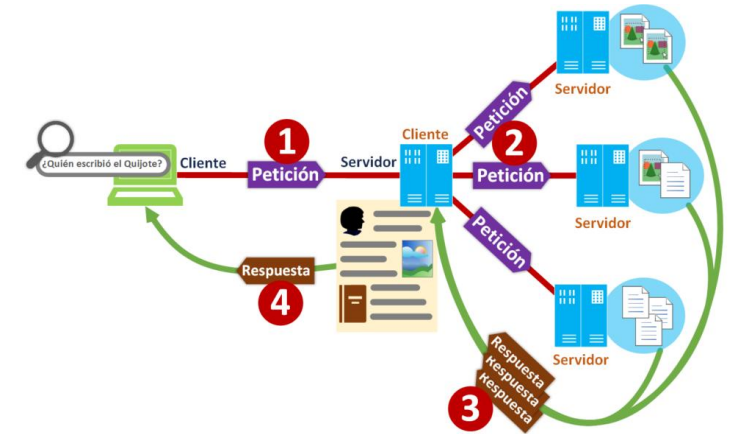
Características de REST:

1. Arquitectura cliente-servidor

- El cliente (ejemplo: navegador, app Angular, Postman) solicita recursos.
- El servidor (ejemplo: Spring Boot) responde con los datos solicitados.

2. Stateless (sin estado)

- Cada petición del cliente incluye toda la información necesaria (autenticación, parámetros).
- El servidor no guarda estado entre una petición y otra.



TRANSFORMACIÓN

3. Operaciones basadas en recursos

- Todo se trata como un **recurso** (ejemplo: un restaurante, un pedido, una mesa).
- Cada recurso se identifica con un **URL único** (ejemplo: /api/restaurantes/1).

4. Métodos HTTP estandarizados

- REST utiliza los métodos del protocolo HTTP para operar sobre recursos:
 - ✓ GET → leer.
 - ✓ POST → crear.
 - ✓ PUT → actualizar.
 - ✓ DELETE → eliminar.

5. Formatos de intercambio de datos

- **JSON** es el formato más usado hoy en día.
- También se pueden usar XML o YAML.
- Ejemplo en JSON de un recurso Restaurante:

```
{  
  "id": 1,  
  "nombre": "Restaurante Central",  
  "direccion": "Av. Grau 123",  
  "capacidad": 50  
}
```

Implementación básica de una API REST con Spring Boot



Estructura de un proyecto típico:

```
com.ejemplo.api
├── controller
├── service
├── repository
└── model
```

Ejemplo de controlador REST:

```
java
@RestController
@RequestMapping("/api/usuarios")
public class UsuarioController {

    @GetMapping
    public List<Usuario> obtenerUsuarios() {
        return usuarioService.getAllUsuarios();
    }

    @PostMapping
    public Usuario crearUsuario(@RequestBody Usuario usuario) {
        return usuarioService.createUsuario(usuario);
    }
}
```

- **@RestController** : Define la clase como controlador REST
- **@RequestMapping**: Ruta base para todas las peticiones
- **@GetMapping, @PostMapping, etc.** : Define el método HTTP usado
- **@RequestBody**: Recibe datos del cuerpo de la petición

¿Qué es HTTP?

- HTTP (HyperText Transfer Protocol) es el protocolo de comunicación que usan los navegadores, apps móviles y clientes de software para **intercambiar** información con un servidor web.
- En el caso de **REST**, HTTP se aprovecha al máximo para manejar **recursos** usando sus métodos estandarizados.

Métodos principales en REST

REST se apoya en cuatro métodos básicos de HTTP para implementar las operaciones CRUD (Create, Read, Update, Delete):

1. GET – Consultar datos (Read)

- Uso: obtener información de un recurso o lista de recursos.

Ejemplo:

- **GET /api/restaurantes** (Devuelve todos los restaurantes.)
- **GET /api/restaurantes/1** (Devuelve los datos del restaurante con ID=1.)

Métodos HTTP y sus usos en la práctica



2. POST – Crear un recurso (Create)

Uso: enviar datos al servidor para crear un nuevo recurso.

Ejemplo:

```
POST /api/restaurantes
Content-Type: application/json
```

```
{
  "nombre": "Restaurante Central",
  "direccion": "Av. Grau 123",
  "capacidad": 50
}
```

Crea un nuevo restaurante en la base de datos.

Características:

- Envía datos en el **cuerpo (body)** de la solicitud.
- Retorna generalmente un **código 201 (Created)**.

3. PUT – Actualizar un recurso (Update)

Uso: actualizar completamente un recurso existente.

Ejemplo:

```
PUT /api/restaurantes/1
Content-Type: application/json
```

```
{
  "nombre": "Restaurante Actualizado",
  "direccion": "Av. Piura 456",
  "capacidad": 60
}
```

Reemplaza los datos del restaurante con ID=1.

Características:

- Se usa para modificaciones **totales** del recurso.
- En muchos proyectos modernos se complementa con **PATCH** (actualización parcial).

TRANSFORMACIÓN

4. DELETE – Eliminar un recurso (Delete)

Uso: Eliminar recurso existente.

Ejemplo:

```
DELETE /api/restaurantes/1
```

Características:

- Una vez eliminado, el recurso no debe estar disponible.
- Generalmente retorna un código **200 (OK)** o **204 (No Content)**.

Métodos HTTP y sus usos en la práctica



Método	Operación	CRUD	Ejemplo	Código esperado
GET	Consultar recurso	Read	/api/restaurantes	200 OK
POST	Crear recurso	Create	/api/restaurantes	201 Created
PUT	Actualizar recurso	Update	/api/restaurantes/1	200 OK
DELETE	Eliminar recurso	Delete	/api/restaurantes/1	204 No Content

Implementación de una API REST en Spring Boot



¿Por qué usar Spring Boot para APIs REST?

- Simplifica la creación de aplicaciones Java listas para producción.
- Ofrece integración directa con Spring Web, Spring Data JPA, y bases de datos como H2/MySQL.
- Permite construir endpoints RESTful de forma rápida, segura y escalable.

Estructura básica de un proyecto Spring Boot para API REST

1. Entidad (@Entity)

Representa la tabla en la base de datos.

```
@Entity
public class Restaurante {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nombre;
    private String direccion;
    private int capacidad;
}
```

Implementación de una API REST en Spring Boot



2. Repositorio (@Repository)

Se encarga del acceso a datos con JPA.

```
public interface RestauranteRepository extends JpaRepository<Restaurante, Long> {  
}
```

3. Servicio (@Service)

Contiene la lógica de negocio.

```
@Service  
public class RestauranteService {  
    @Autowired  
    private RestauranteRepository repository;  
  
    public List<Restaurante> listar() {  
        return repository.findAll();  
    }  
  
    public Restaurante guardar(Restaurante r) {  
        return repository.save(r);  
    }  
}
```

4. Controlador (@RestController)

Expone la API REST a través de endpoints.

Endpoints creados en este ejemplo

- **GET /api/restaurantes** → Lista todos los restaurantes.
- **POST /api/restaurantes** → Crea un nuevo restaurante.
- (Se pueden ampliar con PUT y DELETE en actividades posteriores).

```
@RestController
@RequestMapping("/api/restaurantes")
public class RestauranteController {

    @Autowired
    private RestauranteService service;

    @GetMapping
    public List<Restaurante> getAll() {
        return service.listar();
    }

    @PostMapping
    public Restaurante create(@RequestBody Restaurante restaurante) {
        return service.guardar(restaurante);
    }
}
```

Herramientas de prueba de API REST

¿Por qué probar una API REST?

- Asegura que los endpoints creados funcionan según lo esperado.
- Permite validar el correcto uso de métodos HTTP (GET, POST, PUT, DELETE).
- Detecta errores en la estructura de datos, autenticación o lógica de negocio.
- Facilita la colaboración entre equipos (backend y frontend).

Principales herramientas de prueba

1. Postman (herramienta gráfica)

- Es la herramienta más usada para probar APIs REST.
- Permite:
 - ✓ Enviar solicitudes HTTP con distintos métodos (GET, POST, PUT, DELETE).
 - ✓ Incluir cuerpos JSON en las solicitudes POST/PUT. Revisar respuestas del servidor (código HTTP, body, headers).



POSTMAN

Ejemplo práctico con nuestro proyecto:

- Endpoint: POST <http://localhost:8080/api/restaurantes>
- Body JSON:

```
{  
  "nombre": "Restaurante Central",  
  "direccion": "Av. Grau 123",  
  "capacidad": 60  
}
```

Respuesta esperada: código **201 (Created)** + objeto JSON con ID generado.

TRANSFORMACIÓN

Herramientas de prueba de API REST



2. cURL

- Línea de comandos
- Muy útil en servidores o scripts automáticos



3. Insomnia

- Alternativa ligera a Postman, ideal para debugging



4. Otras herramientas:

- Swagger (OpenAPI) para documentación interactiva
- JUnit + MockMvc para pruebas automatizadas en Spring



Implementación de API REST – Métodos HTTP y pruebas con Postman

Objetivo de la actividad

Que los estudiantes implementen un CRUD básico en Spring Boot para la gestión de mesas en un restaurante, aplicando correctamente los métodos HTTP (GET, POST, PUT, DELETE) y validando su funcionamiento mediante Postman.

1. Crear la entidad Mesa

Definir una clase Java con los siguientes atributos:

- id (Long)
- numero (int) – número de la mesa
- capacidad (int) – cantidad de personas
- disponible (boolean) – indica si la mesa está libre u ocupada

2. Configurar capas del proyecto

Organizar el proyecto siguiendo buenas prácticas:

- model: clase Mesa.repository: interfaz Mesa
- Repository que extienda JpaRepository.

- service: clase MesaService con métodos CRUD.
- controller: clase MesaController con endpoints REST.

3. Implementar los endpoints RESTful

- Listar todas las mesas (GET): GET /api/mesas
- Registrar una nueva mesa (POST): POST /api/mesas
- Actualizar una mesa existente (PUT): PUT /api/mesas/{id}
- Eliminar una mesa (DELETE): DELETE /api/mesas/{id}

4. Probar con Postman

- Crear una colección Postman con las cuatro operaciones.
- Capturar evidencias de:
 - ✓ Solicitud enviada.
 - ✓ Respuesta recibida (código HTTP + JSON).



Actividad Práctica



1. POST /api/mesas

json

```
{
  "numero": 1,
  "capacidad": 4,
  "disponible": true
}
```

→ Respuesta:

json

```
{
  "id": 1,
  "numero": 1,
  "capacidad": 4,
  "disponible": true
}
```

2. GET /api/mesas

json

```
[
  {
    "id": 1,
    "numero": 1,
    "capacidad": 4,
    "disponible": true
  }
]
```

3. PUT /api/mesas/1

json

```
{
  "numero": 1,
  "capacidad": 6,
  "disponible": false
}
```

→ Respuesta:

json

```
{
  "id": 1,
  "numero": 1,
  "capacidad": 6,
  "disponible": false
}
```

4. DELETE /api/mesas/1

- Respuesta: 204 No Content

EJERCICIO DE PRÁCTICA

Objetivo de la actividad

Que el estudiante implemente una API REST avanzada en Spring Boot que gestione Restaurantes y Mesas con relación One-to-Many, aplicando validaciones de datos, manejo de errores y probando la API con Postman.

1. Diseñar dos entidades relacionadas:

- Restaurante (id, nombre, dirección, capacidad total).
- Mesa (id, número, capacidad, disponible, restaurante_id).
- Relación: Un restaurante tiene muchas mesas.

2. Configurar repositorios y servicios:

- RestauranteRepository y MesaRepository con Spring Data JPA.
- Servicios que permitan gestionar ambas entidades.

3. Implementar los endpoints RESTful:

• Restaurantes

- ✓ GET /api/restaurantes → Listar restaurantes.
- ✓ POST /api/restaurantes → Registrar un restaurante.

• Mesas

- ✓ GET /api/restaurantes/{id}/mesas → Listar las mesas de un restaurante.
- ✓ POST /api/restaurantes/{id}/mesas → Registrar una mesa asociada a un restaurante.
- ✓ PUT /api/mesas/{id} → Actualizar datos de una mesa.
- ✓ DELETE /api/mesas/{id} → Eliminar una mesa.

4. Aplicar validaciones:

- Un restaurante debe tener nombre obligatorio y capacidad > 0.
- Una mesa debe tener capacidad mínima de 2.
- Si se intenta registrar una mesa en un restaurante inexistente, retornar error 404 Not Found.

5. Manejo de excepciones personalizadas:

Crear una clase GlobalExceptionHandler con @ControllerAdvice para devolver errores en formato JSON.

6. Pruebas con Postman:

Crear una colección de pruebas:

- Registrar un restaurante.
- Registrar varias mesas asociadas al restaurante.
- Listar mesas de un restaurante.
- Intentar registrar mesa en restaurante inexistente → verificar error 404.



Actividad Práctica



```
src/main/java/com/example/resto
├─ RestoApplication.java
├─ controller
│   ├── RestauranteController.java
│   └─ MesaController.java
├─ dto
│   ├── CreateMesaRequest.java
│   └─ CreateRestauranteRequest.java
├─ exception
│   ├── NotFoundException.java
│   └─ GlobalExceptionHandler.java
├─ model
│   ├── Restaurante.java
│   └─ Mesa.java
├─ repository
│   ├── RestauranteRepository.java
│   └─ MesaRepository.java
└─ service
    ├── RestauranteService.java
    └─ MesaService.java
```

EJERCICIO DE PRÁCTICA

- **TEMA 1:** Plataforma inteligente para gestión de mantenimiento predictivo de flotas
- **TEMA 2:** Sistema de trazabilidad de productos de la cadena de frío
- **TEMA 3:** Plataforma de gestión de incidentes de infraestructura urbana
- **TEMA 4:** Sistema de gestión de residuos electrónicos
- **TEMA 5:** Plataforma de gestión de servicios técnicos especializados
- **TEMA 6:** Plataforma de gestión de riesgos y seguridad operacional
- **TEMA 7:** Plataforma de gestión de producción agrícola
- **TEMA 8:** Sistema Web para la Gestión y Operación de Tours Turísticos Marítimos
- **TEMA 9:** Sistema Web para la Gestión, Seguimiento en una Entidad Pública



cierre



¿Que hemos aprendido hoy?

1. REST (Representational State Transfer) es un estilo de arquitectura que define un conjunto de reglas para construir servicios web escalables y desacoplados.
2. Postman es una Interfaz gráfica para enviar solicitudes HTTP, que permite guardar colecciones, automatizar pruebas, así como ver fácilmente headers, body y status code.

CIERRE



**Universidad
Tecnológica
del Perú**